

LAMPIRAN B - FUNGSI DAN CARA KERJA SETIAP FITUR

Sistem Solar Dryer IoT - Monitoring & Kontrol Pengering Surya

1. ARSITEKTUR SISTEM

Sistem Solar Dryer IoT terdiri dari 3 komponen utama:

No	Komponen	Teknologi	Fungsi
1	Firmware ESP32	Arduino C++	Membaca sensor, kontrol blower, logging SD Card
2	Web Dashboard	React.js	Monitoring & kontrol via browser
3	Mobile App	Flutter (Dart)	Monitoring & kontrol via Android
4	Backend	Firebase RTDB	Database real-time, autentikasi

2. FITUR FIRMWARE ESP32 (smart_ac.ino)

2.1 Pembacaan Sensor DHT22 (10 Unit)

- **Fungsi:** `readAllSensors()`
- **Cara Kerja:** Membaca suhu dan kelembaban dari 10 sensor DHT22 yang terhubung ke pin GPIO (32, 33, 25, 27, 26, 14, 17, 13, 16, 4). Data dibaca setiap 60 detik. Sensor yang gagal dibaca ditandai sebagai NaN dan tidak diikutsertakan dalam perhitungan rata-rata.
- **Output:** Rata-rata suhu (`avgTemp`) dan kelembaban (`avgHum`) dari sensor yang valid.

2.2 Kontrol Blower PWM & Mode Otomatis (BTS7960)

- **Fungsi:** `setBlowerSpeed(idx, speedPct)` dan `autoControlBlower()`
- **Cara Kerja:**
 - **Mode Manual:** Menerima perintah kecepatan (0-100%) dari Firebase Realtime Database melalui stream callback.
 - **Mode Otomatis:** Menyesuaikan kecepatan berdasarkan rata-rata suhu sensor secara real-time. Rentang logika:
`<35°C` (OFF/0%), `35-41.9°C` (25%), `42-48.9°C` (50%), `49-54.9°C` (75%), `≥55°C` (100%).
- Pin Enable (EN) diaktifkan saat kecepatan > 0 dan dimatikan saat 0 untuk menghemat daya. Sinyal PWM menggunakan fungsi `analogWrite()` .

2.3 Sinkronisasi Firebase

- **Fungsi:** `initFirebase()` , `fanStreamCallback()` , `pushHistory()`
- **Cara Kerja:**
 - **Upload Real-time:** Data sensor dikirim ke node `/sensor_data` setiap pembacaan sensor.
 - **Stream Listener:** ESP32 mendengarkan perubahan pada node `/blowers` untuk menerima perintah kecepatan blower.
 - **Push History:** Data historis dikirim ke node `/history` setiap 60 detik dengan server timestamp.
- **Autentikasi:** Menggunakan Email/Password Firebase Auth.

2.4 Logging ke SD Card

- **Fungsi:** `logToSD(now)`

- **Cara Kerja:** Data sensor dicatat dalam format CSV ke file `/data.csv` di SD Card. Header otomatis dibuat saat file belum ada. Pencatatan dilakukan setiap menit (cek berdasarkan `lastLoggedMinute`).
- **Format:** `Tanggal,Waktu,T1,H1,T2,H2,...,T10,H10`

2.5 Manajemen Koneksi WiFi

- **Fungsi:** `connectWiFi()`
- **Cara Kerja:** ESP32 mencoba terhubung ke WiFi dengan timeout 20 detik. Jika terputus di loop utama, sistem akan menunggu auto-reconnect bawaan ESP32 setiap 15 detik.

2.6 RTC DS3231 (Real-Time Clock)

- **Cara Kerja:** Memberikan timestamp akurat untuk pencatatan data. Waktu RTC disinkronkan ke system time ESP32 via `settimeofday()` .

3. FITUR WEB DASHBOARD (React.js)

3.1 Autentikasi & Login (`LoginPage.jsx` , `AuthContext.jsx`)

- **Cara Kerja:** Menggunakan Firebase Authentication dengan Email/Password. Setelah login berhasil, data user disimpan ke `/users/{uid}` dan role default "guess" ditetapkan di `/user_akses/{uid}` . Session berlaku 1 jam dengan auto-logout.
- **Presence System:** Status online/offline user dikelola via node `/presence/{uid}` dengan `onDisconnect()` otomatis menghapus saat koneksi terputus.

3.2 Role-Based Access Control / RBAC (`AuthContext.jsx`)

- **5 Level Role:**

Role	Level	Akses Tab
<code>guess</code>	0	Dashboard saja
<code>view_data</code>	1	Dashboard + Data
<code>eksekusi_data</code>	2	Dashboard + Control + Data
<code>report_data</code>	3	Dashboard + Data + Export
<code>admin</code>	4	Semua (termasuk Management)

- **Cara Kerja:** Fungsi `canAccessTab(role, tabId)` memeriksa apakah role user memiliki akses ke tab tertentu. Tab yang tidak dapat diakses disembunyikan dari navigasi.

3.3 Dashboard Monitoring (`App.jsx` - Tab Dashboard)

- **Cara Kerja:** Menampilkan data real-time dari Firebase node `/sensor_data` . Terdapat:
 - **Info Bar:** Tanggal dan waktu pembacaan terakhir dari ESP32.
 - **Average Cards:** 2 kartu rata-rata suhu dan kelembaban (`SensorCard`).
 - **Individual Sensor Grid:** Grid 10 sensor individual (`IndividualSensorCard`) menampilkan suhu, kelembaban, dan status aktif/tidak aktif per sensor.
 - **Grafik Tren:** Chart garis menggunakan Recharts (`SensorChart`) menampilkan tren historis rata-rata suhu dan kelembaban.
- **Status Koneksi:** Sistem dianggap online jika data terakhir diterima dalam 120 detik.

3.4 Kontrol Blower & Auto Mode (`App.jsx` - Tab Control, `BlowerControl.jsx`)

- **Cara Kerja:**
 - Menampilkan status 2 blower (aktif/mati, kecepatan %).

- **Mode Switch:** Toggle antara kontrol “Otomatis” dan “Manual”. Mode Otomatis akan mematikan interaksi slider (read-only) dan menampilkan tabel referensi suhu.
- **Master Blower Control:** Slider PWM (0-100%) untuk pengaturan manual.
- Nilai kecepatan ditulis ke Firebase node `/blowers/blower1/speed` . Mode kontrol (auto/manual) ditulis ke `/control_mode` .
- Estimasi RPM ditampilkan (kecepatan × 6000 RPM).
- Notifikasi toast ditampilkan setelah perubahan berhasil/gagal.

3.5 Riwayat Data Sensor (`App.jsx` - `Tab Data`)

- **Cara Kerja:** Menampilkan tabel data historis dari Firebase node `/history` dengan fitur:
 - **Search:** Filter berdasarkan tanggal/waktu.
 - **Pagination:** Menampilkan 5/10/20/50/100/semua data per halaman.
 - **Tabel Detail:** Menampilkan waktu, rata-rata suhu/kelembaban, dan data individual 10 sensor.
 - **Export CSV:** (Hanya role `report_data` dan `admin`) Mengekspor data ke file CSV. Di platform native (Android), menggunakan Capacitor Filesystem + Share API. Di web, menggunakan Blob download.

3.6 Management User (`ManagementPage.jsx`)

- **Cara Kerja** (Hanya Admin):
 - Menampilkan statistik: Total Users, Users Online, Admins, Active Roles.
 - Tabel daftar user dengan email, status online/offline, current role, last login.
 - **Ubah Role:** Admin dapat mengubah role user lain via dropdown select.
 - **Hapus Akses:** Menghapus data `/user_akses/{uid}` sehingga user kembali ke role “guess”.
 - **Search:** Filter user berdasarkan email, nama, atau role.

3.7 Komponen SensorCard (`SensorCard.jsx`)

- **SensorCard:** Menampilkan rata-rata suhu/kelembaban dengan ikon, nilai numerik, badge “Live”, dan progress bar.
- **IndividualSensorCard:** Menampilkan data per sensor dengan indikator aktif (hijau) / tidak aktif (abu-abu).

3.8 Komponen SensorChart (`SensorChart.jsx`)

- **Cara Kerja:** Menggunakan library Recharts untuk menampilkan LineChart dengan 2 garis (suhu oranye, kelembaban biru). Tooltip custom menampilkan nilai pada hover.
-

4. FITUR MOBILE APP (Flutter)

4.1 Entry Point (`main.dart`)

- Inisialisasi Firebase dan menjalankan `DashboardScreen` . Tema dark mode dengan Material3 dan Google Fonts Inter.

4.2 Dashboard Screen (`dashboard_screen.dart`)

- **Cara Kerja:** Mendengarkan stream data dari Firebase untuk sensor, blower, dan history. Menampilkan:
 - SensorCard untuk suhu dan kelembaban.
 - BlowerControl untuk setiap blower.
 - SensorChartWidget untuk grafik historis.
 - Indikator koneksi online/offline (heartbeat timer 5 detik, threshold 120 detik).

4.3 Firebase Service (`firebase_service.dart`)

- **Cara Kerja:** Menyediakan stream untuk `sensor_data` , `blowers` , dan `history` (limited 1000 entri terakhir). Method `setBlowerSpeed()` menulis kecepatan ke Firebase.

4.4 Blower Control Widget (`blower_control.dart`)

- **Cara Kerja:** Slider untuk mengatur kecepatan blower (0-100%). Menampilkan estimasi RPM (kecepatan × 2480). Nilai dikirim ke Firebase saat slider dilepas.

4.5 Chart Widget (`chart_widget.dart`)

- **Cara Kerja:** Menggunakan library `fl_chart` untuk menampilkan grafik garis suhu dan kelembaban. Tooltip menampilkan nilai saat disentuh.

4.6 Sensor Card Widget (`sensor_card.dart`)

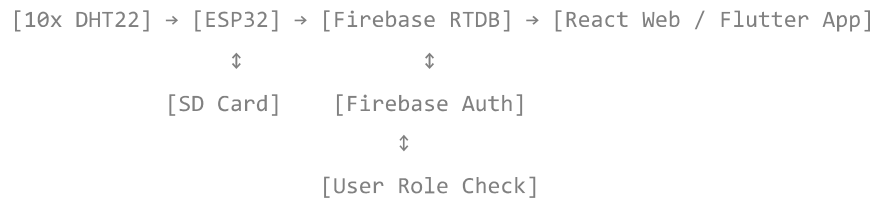
- **Cara Kerja:** Menampilkan nilai sensor dengan warna status dinamis:
 - Suhu: Hijau ($\leq 28^{\circ}\text{C}$), Oranye ($28-35^{\circ}\text{C}$), Merah ($> 35^{\circ}\text{C}$)
 - Kelembaban: Hijau ($\leq 60\%$), Oranye ($60-80\%$), Merah ($> 80\%$)
-

5. FIREBASE REALTIME DATABASE RULES (`database.rules.json`)

Node	Read	Write	Keterangan
<code>/sensor_data</code>	Public	Public	Data sensor ESP32
<code>/blowers</code>	Public	Public	Kontrol blower
<code>/control_mode</code>	Public	Public	Mode kontrol
<code>/history</code>	Public	Public	Data historis
<code>/users</code>	Admin only	Self only	Profil user

Node	Read	Write	Keterangan
<code>/user_akses</code>	Admin only	Admin / new user	Role management

6. ALUR DATA SISTEM (Data Flow)



1. ESP32 membaca 10 sensor DHT22 setiap 60 detik.
2. Data rata-rata dan detail sensor dikirim ke Firebase `/sensor_data`.
3. Data historis di-push ke Firebase `/history` setiap 60 detik.
4. Data juga dicatat ke SD Card dalam format CSV.
5. Web Dashboard dan Mobile App mendengarkan perubahan data secara real-time.
6. User mengontrol blower via dashboard → data ditulis ke `/blowers` → ESP32 menerima via stream → mengatur PWM motor.